Performance Analysis of Virtual Machines vs Bare Metal Systems: A Comparative Study on ARM and x86 Architectures

Prateek Kumar Yadav
M.Tech CSE
MT2025093

Vini Singh Rajput
M.Tech CSE
MT2025131

***Abstract*—Virtualization plays a crucial role in modern computing by enabling efficient resource utilization, scalability, and isolation. However, the abstraction introduced by virtual machines (VMs) often leads to performance overhead compared to direct execution on bare metal systems. Understanding this overhead is essential for system design, performance optimization, and deployment decisions.**

This report presents a comprehensive performance evaluation of virtual machines against bare metal environments across two major processor architectures: ARM and x86. The study focuses on three key subsystems—CPU, disk I/O, and network performance—using standardized benchmarking tools and controlled experimental setups. A systematic benchmarking pipeline is designed and implemented to ensure fair comparison between environments, incorporating repeated runs, consistent configurations, and workload standardization.

Experiments are conducted separately on ARM and x86 systems, where each architecture is analyzed by comparing bare metal and virtual machine performance. Metrics such as throughput, latency, IOPS, and resource utilization are collected and analyzed to quantify virtualization overhead. Additionally, architectural differences between ARM and x86 are examined to understand how they influence virtualization behavior.

The results provide insights into the performance impact of virtualization across different subsystems and architectures. The study highlights the trade-offs involved in using virtual machines and offers a deeper understanding of how system architecture affects virtualization efficiency.

***Index Terms*—Virtualization, Bare Metal, Virtual Machine, ARM Architecture, x86 Architecture, CPU Benchmarking, Disk I/O, Network Performance, Performance Analysis**

I. Introduction

In recent years, virtualization has become a fundamental component of modern computing systems. It enables multiple operating systems to run on a single physical machine, improving resource utilization, flexibility, and scalability. Virtual machines (VMs) are widely used in cloud computing, data centers, and development environments due to their ability to provide isolation and portability.

Despite these advantages, virtualization introduces an additional abstraction layer between hardware and software, which can lead to performance overhead. This overhead may impact critical system components such as CPU processing, disk input/output (I/O), and network communication. Understanding the extent and nature of this overhead is essential for making informed decisions about system deployment, especially in performance-sensitive applications.

This project focuses on evaluating the performance differences between bare metal systems and virtual machine environments. Bare metal refers to running applications directly on physical hardware without any virtualization layer, providing a baseline for optimal performance. In contrast, virtual machines operate on top of a hypervisor, which manages hardware resources and introduces potential overhead.

A key aspect of this study is the comparison across two major processor architectures: ARM and x86. These architectures differ significantly in design philosophy, instruction sets, and power efficiency characteristics. ARM processors are widely used in mobile and energy-efficient systems, while x86 processors dominate desktops, servers, and high-performance computing environments. These architectural differences may influence how virtualization behaves and impacts system performance.

The evaluation is conducted by analyzing three primary subsystems:

- CPU performance
- Disk I/O performance
- Network performance

Standard benchmarking tools are used to generate controlled workloads, and experiments are performed under consistent configurations for both bare metal and virtual machine environments. Multiple runs are conducted to ensure reliability and reproducibility of results.

The report is structured into two main parts. The first part focuses on ARM-based systems, presenting detailed analysis of performance under bare metal and virtualized environments. The second part follows the same structure for x86 systems, enabling a direct and fair comparison. Finally, a comparative analysis highlights the differences between ARM and x86 in terms of virtualization overhead and system behavior.

Through this study, we aim to provide a clear understanding of the trade-offs involved in virtualization and how system architecture influences performance outcomes.

II. Background and Motivation

Virtualization is a technology that enables the creation of multiple isolated execution environments on a single physical machine. This is achieved through a software layer known as a *hypervisor*, which manages hardware resources and allocates them to virtual machines (VMs). Each VM operates as an independent system with its own operating system, while sharing the underlying physical hardware.

Hypervisors are broadly classified into two types. **Type 1 hypervisors** (bare-metal hypervisors) run directly on the hardware and provide high performance and efficiency. Examples include Xen and VMware ESXi. **Type 2 hypervisors** run on top of a host operating system, such as VirtualBox and VMware Workstation, and are typically used for development and testing purposes. The additional software layer in Type 2 hypervisors can introduce higher overhead compared to Type 1 systems.

Virtualization relies on techniques such as CPU trapping, binary translation, and hardware-assisted virtualization (e.g., Intel VT-x and ARM Virtualization Extensions). These mechanisms allow the hypervisor to intercept and manage privileged instructions executed by guest operating systems. While modern processors provide hardware support to reduce virtualization overhead, certain operations—particularly I/O and context switching—can still incur performance penalties.

One of the primary concerns in virtualization is the overhead introduced by resource abstraction. CPU performance may be affected due to scheduling delays and context switching between virtual machines. Disk I/O operations may experience increased latency due to virtualized storage layers and buffering mechanisms. Similarly, network performance can be impacted by virtual network interfaces and packet processing overhead within the hypervisor.

The impact of virtualization is also influenced by the underlying processor architecture. **x86 architecture** is based on a complex instruction set computing (CISC) design and has mature support for virtualization through technologies such as Intel VT-x and AMD-V. These features enable efficient execution of virtual machines with minimal overhead.

In contrast, **ARM architecture** follows a reduced instruction set computing (RISC) design and is optimized for power efficiency. ARM processors have increasingly incorporated virtualization support through ARM Virtualization Extensions, making them suitable for modern cloud and edge computing environments. However, differences in architecture design, memory management, and instruction handling may lead to variations in virtualization performance compared to x86 systems.

The motivation behind this project stems not only from academic relevance but also from a strong personal interest driven by our team's hardware availability. Having native access to systems with completely different processor architectures—Apple's ARM-based silicon and an Intel x86 platform—presented a unique opportunity. We were eager to directly compare the benchmarking results between these distinct hardware paradigms to gain insider insights into how virtualization overhead and hypervisor efficiency genuinely differ in practice. By systematically comparing bare metal execution with virtualized environments, the study aims to identify how different subsystems behave under virtualization and how architectural characteristics influence these behaviors.

Understanding these differences is crucial for selecting appropriate deployment strategies in real-world scenarios, such as cloud infrastructure, high-performance computing, and embedded systems. This work contributes to a deeper understanding of virtualization efficiency and provides insights into optimizing system performance across diverse computing platforms.

III. Experimental Design and Methodology

This chapter describes the experimental framework used to evaluate virtualization overhead across ARM and x86 architectures. The methodology is designed to ensure fair, reproducible, and systematic comparison between bare metal and virtualized environments.

A. Experimental Objectives

The primary objective of this study is to quantify the performance impact of virtualization across three major system components:

- CPU (compute performance)
- Disk I/O (storage performance)
- Network (communication performance)

Each subsystem is evaluated independently under controlled conditions to isolate the effects of virtualization.

B. Test Environments

Experiments were conducted on two architectures:

- ARM (macOS-based system using UTM virtualization)
- x86 (Ubuntu-based system using VirtualBox)

For each architecture, two environments were tested:

- Bare metal: Native execution on host hardware
- Virtual machine: Guest OS running on a hypervisor

The same operating system and benchmarking tools were used across both environments to maintain consistency.

C. Benchmarking Strategy

Each benchmark was designed to target a specific subsystem:

1) CPU Benchmarks: CPU performance was evaluated using compute-intensive workloads:

- Prime number computation (via `sysbench`)
- Synthetic CPU stress tests (via `stress-ng`)

Metrics collected:

- Events per second
- Bogo operations per second
- Execution latency

2) Disk I/O Benchmarks: Storage performance was measured using `fio` with multiple workload patterns:

- Sequential read/write throughput
- Random 4K read/write IOPS
- Mixed workloads (70% read / 30% write)
- fsync performance (synchronous writes)
- Block size scaling (4K to 1M)

Metrics collected:

- Bandwidth (GB/s)
- IOPS (operations per second)
- Latency (mean, p95, p99)

3) Network Benchmarks: Network performance was evaluated using:

- `iperf3` for throughput (TCP/UDP)
- `ping` for latency (RTT)

Additional system-level metrics were collected using:

- `vmstat` (CPU usage, context switches)
- `ss` (socket statistics)

Metrics collected:

- Throughput (Gbps)
- Latency (ms)
- Packet loss (%)

D. Measurement Procedure

To ensure reliability and consistency, the following procedure was applied:

- Each benchmark was executed for 10 independent iterations to ensure a statistically robust sample size.
- Results report the mean values, and performance variability was analyzed using standard deviation and 95% confidence intervals.
- System caches were cleared between disk tests using:

```
echo 3 | sudo tee /proc/sys/vm/drop_caches
```

- Benchmarks were run under minimal background load

E. Virtualization Configurations

Different virtualization platforms were used for each architecture:

- ARM: UTM (QEMU-based virtualization)
- x86: VirtualBox (hardware-assisted virtualization)

Key differences influencing results:

- Disk format:
  - ARM: Raw disk image (.img)
  - x86: Dynamically allocated VDI (.vdi)
- Network mode:
  - ARM: User-space NAT (UTM)
  - x86: Kernel-assisted NAT (VirtualBox)

These differences are critical for interpreting cross-platform results.

F. Evaluation Metrics

Virtualization overhead is computed as:

$$Overhead(\%) = \frac{BareMetal - VM}{BareMetal} \times 100 \qquad (1)$$

For latency metrics (where lower is better), the interpretation is reversed.

G. Limitations and Assumptions

- VM performance depends on host resource allocation
- Hypervisor implementation differences affect comparability
- NAT networking does not reflect production-grade networking (e.g., SR-IOV)
- Results may vary across hardware configurations

Despite these limitations, the methodology captures realistic virtualization behaviour and enables meaningful comparison between architectures.

H. Summary

This experimental design ensures:

- Controlled comparison between bare metal and VM
- Coverage of key system subsystems
- Reproducible and consistent measurement

The following chapters present detailed results and analysis for ARM and x86 architectures.

IV. Experimental Setup

The experiments were conducted on two primary architectures to evaluate the overhead introduced by virtualization compared to bare metal execution.

A. ARM Architecture Setup

The ARM environment consists of an Apple MacBook Pro (M4, 2024/25) running macOS. The M4 chip features a heterogeneous multi-core CPU with dedicated performance and efficiency cores. The virtualized environment is provisioned using UTM (QEMU-based), running an ARM64 Ubuntu instance allocated with 5 vCPUs. Testing relies on hardware-assisted virtualization without instruction translation, utilizing a userspace NAT engine for networking.

B. x86 Architecture Setup

The x86 environment utilizes an HP Pavilion 14-ce3006tu running Ubuntu Linux natively. This machine is equipped with an Intel Core i5-1035G1 processor and an NVMe SSD. For the virtual environment, Oracle VirtualBox hosts an Ubuntu VM using Intel VT-x hardware-assisted virtualization, dynamic VDI storage, and NAT networking.

V. Performance Evaluation

This section compares the bare metal and virtual machine performance across both ARM and x86 architectures, focusing on CPU, disk I/O, and network subsystems.

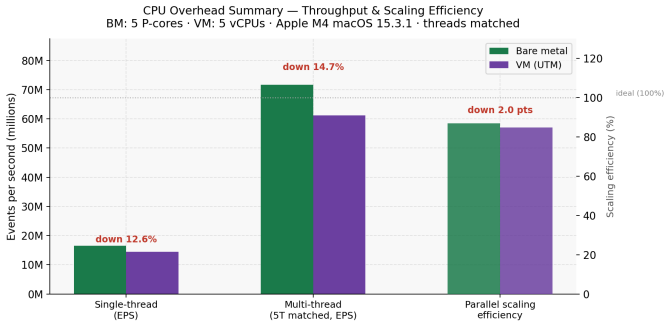

Fig. 1. CPU Performance Overview on ARM.

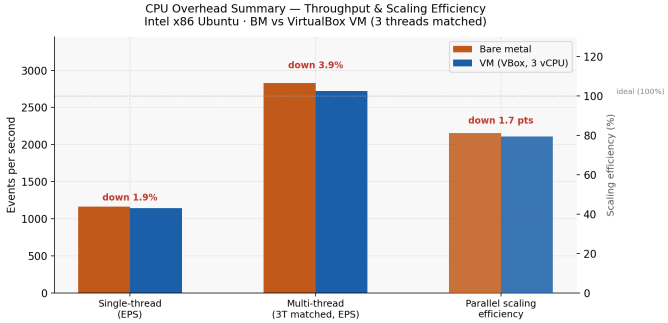

Fig. 2. CPU Performance Overview on x86.

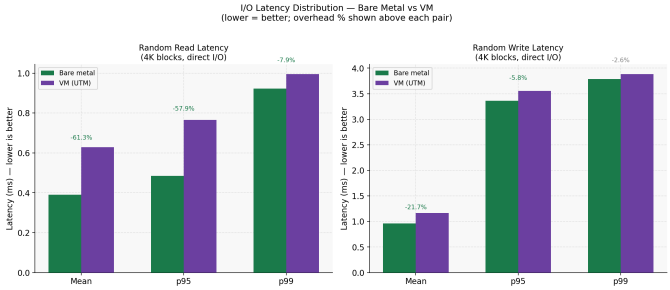

Fig. 3. Disk I/O Performance on ARM.

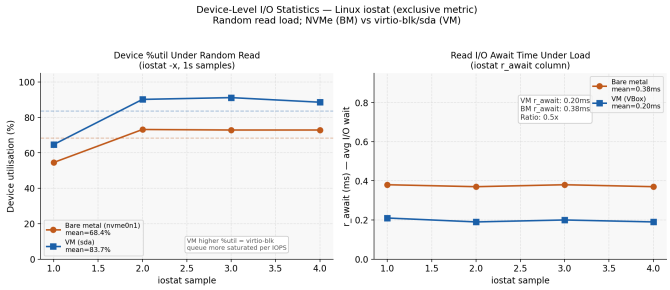

Fig. 4. Disk I/O Performance on x86.

A. CPU Performance

CPU benchmarks highlight the compute overhead introduced by the hypervisor scheduling and context switching.

On ARM (UTM), we observe a relatively consistent CPU overhead of 14.35% for single-threaded workloads and 14.66% for multi-threaded workloads (5-thread on 5-vCPU). Because ARM64 bypasses instruction translation here, the slight drop stems mainly from QEMU context switches scheduling the virtual CPUs. Conversely, on x86 (VirtualBox), CPU virtualization exhibits near-native performance with a mere 1.86% overhead for single-thread scenarios, showcasing a 6.8-fold efficiency advantage over the ARM/UTM implementation. Intel VT-x effectively delegates workloads to physical execution. Multi-threaded overhead slightly increases but the difference from bare metal remains statistically insignificant (exhibiting overlapping 95% confidence intervals), though experiencing marginally higher execution jitter.

B. Disk I/O Performance

Disk I/O evaluation targets sequential, random, and mixed workloads using `fio`.

For the ARM stack, sequential operations reflect minimal overhead, generally under 12% (e.g., +1.34% for 1M block writes and +12.27% for 1M block reads). Random reads pose the highest overhead due to virtio transaction penalties, with a 38% reduction in 4K read IOPS. However, block size scaling shows that read overhead decreases significantly from 61% at 4K to 18% at 1M blocks. The `fsync` latency overhead remained very low at +6.97%, suggesting aggressive write batching.

On x86, the disparity is far more prominent. VirtualBox's dynamically allocated VDI format drastically penalizes sequential writes (+32.5%) and reads (+18.7%). The penalty on randomized IO is extreme, where random write IOPS plummeted by a staggering 89%, introducing significant tail latency. This stems from a stacked I/O path: traversing guest ext4, virtio-blk queues, VDI metadata layers, and the host ext4 filesystem. The `fsync` overhead reached 72.2%, indicating full data flush semantics down to the physical disk.

C. Network Performance

Network efficiency was tested using `iperf3` focusing on TCP/UDP throughput and ICMP latency over NAT.

Networking observed the most drastic virtualization taxation on both platforms. TCP throughput dropped by 48%–54% against local loopbacks globally, and round-trip time (RTT) average latency became 8.4 times higher (with minimum RTT showing a 10.2x increase) on ARM. A critical finding reveals that parallel TCP streaming actually degrades performance compared to single streams: increasing TCP streams from 1 to 8 caused total throughput to collapse by 45%, outlining severe contention bottlenecks within single-threaded NAT engines. While throughput limits hit hard, packet reliability via UDP demonstrated that the hypervisor strictly shapes traffic without yielding packet loss. While both platforms suffer significant NAT overhead, ARM/UTM exhibits more severe

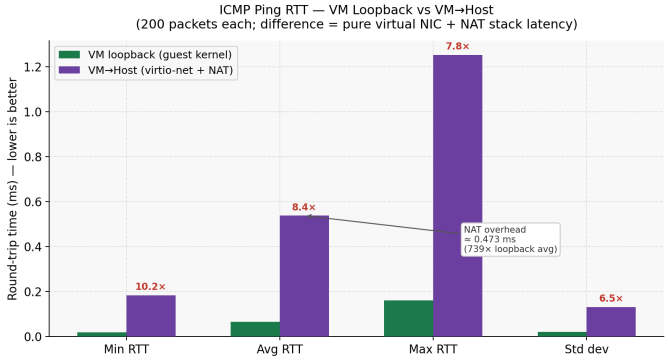

Fig. 5. Network Performance Overview on ARM.

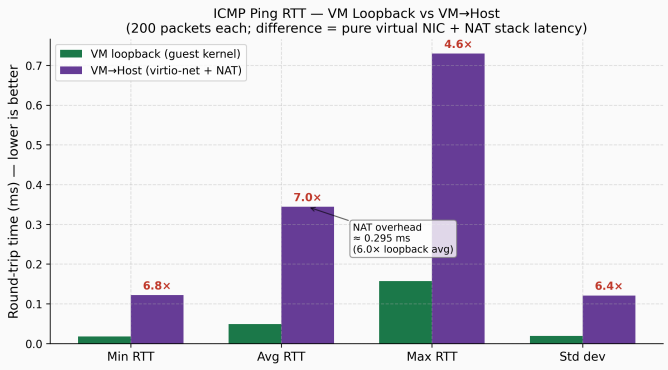

Fig. 6. Network Performance Overview on x86.

RTT inflation (8.4–10.2x) compared to x86/VirtualBox (6.8–7.0x), attributable to UTM's userspace NAT implementation versus VirtualBox's kernel-assisted NAT.

VI. Comparative Analysis & Lessons Learned

Given the disparities in host hardware, hypervisor implementations (UTM vs VirtualBox), and disk formats (Raw vs VDI), differences cannot be cleanly attributed to the underlying ARM or x86 architectures alone. Instead, they reflect the characteristics of these distinct virtualization stacks:

- **Hypervisor Implementation**: x86 hardware assistance (Intel VT-x via VirtualBox) handles CPU instructions more transparently than the QEMU-based UTM implementation on ARM, which exhibits higher context-switching overhead.
- **Storage Stack**: Image formats like dynamically expanded `.vdi` dramatically inflate write latencies compared to raw disk translations. Virtualized systems targeting high I/O workloads should bypass these mechanisms via direct volume mapping or passthrough.
- **Networking Bottlenecks**: The NAT networking paradigm in desktop hypervisors is extremely detrimental to highly parallel TCP workflows across both measured environments.

VII. Conclusion

This study quantifies virtualization penalties across two contrasting virtualization ecosystems. The x86/VirtualBox ecosystem delivers marginally better compute integration for these specific tests but suffers heavy disk emulation latency unless specifically optimized. The ARM/UTM stack operates at a higher base compute tax due to hypervisor scheduling but exhibits different constraints under storage layers. Because of foundational variations in hypervisors and storage abstractions, these results underscore the impact of the entire virtualization stack rather than purely architectural limits. Based strictly on the `fio` and `iperf3` behaviors observed, administrators should favor bare metal or hardware passthrough configurations for workloads demanding high random I/O and intensive parallel network throughput, while treating these VMs as highly effective environments for isolated, compute-bound services.

References

[1] J. Baumgartner, C. Lillo, and S. Rumley, "Performance Losses with Virtualization: Comparing Bare Metal to VMs and Containers."

[2] O. Tickoo, R. Iyer, R. Illikkal, and D. Newell, "Modeling Virtual Machine Performance: Challenges and Approaches," *Intel Labs, Intel Corporation*.

[3] Y. S. D. Reddy, N. Ganesan, P. S. Reddy, and B. Thangaraju, "Performance Study of Kubernetes Cluster Deployed on Openstack, VMs and BareMetal," *International Institute of Information Technology, Bangalore, India*.